2026 FPGA 赛道 · 赛题二

# PMUX 优化设计报告

## Yosys 0.69 / v21-fast 初稿

算法原理 • 整体架构 • 模块职责 • 测试方案

> 当前九例面积平均下降 20.0619%，实际 Windows 网表十八项 EQY 通过。效率结论依赖已披露的复核口径；test1 首轮异常及交付边界尚未完全解决。本稿不写赛项整体通过。

版本日期：2026-10-27

团队姓名、真实贡献与现场职责：待成员核实填写

源码身份：0231002c240a080e…885599eb43678

本 PDF 根据经复核的 Markdown 初稿排版，保留既有不利样本、规则解释和未完成项。后续版本须更新身份与证据，不能套用本稿成绩。

# 目录

1. 任务目标、验收依据与当前结论 3

2. 整体架构与综合调用流程 4

3. 条件事实提取与数据替换 5

4. 跨 PMUX 分支的运算共享 6

5. 完整默认表与跨输出共享决策图 8

6. 安全条件、未覆盖语义与实现边界 9

7. 功能验证方案与证据链 10

8. 性能测量设计、当前成绩与异常 11

9. 交付、复现方式与尚待完善项 13

10. 三人实际分工与答辩核对（待团队填写） 15

附录：主要文件与阅读顺序 15

阅读提示：第 3～5 节解释算法；第 6 节列出保护边界；第 7～8 节区分功能证明与性能证据；第 9～10 节说明交付缺口和待填写分工。

版本：2026-10-27，v21-fast；补充同日 Linux/WSL 独立证明包的迁移复现结果。本文是设计报告初稿，供团队核实，不表示赛项全部验收完成。

本文将“源码实现事实”“已有实测证据”和“尚未解决的问题”分别说明。算法说明依据 521 行的 `selected-v21-fast.cc`，其 SHA-256 为 `0231002c240a080e7eebe1734b9dab49f2db78159c38a68769c885599eb43678`。部署时该文件对应仓库 `src/pmux_competition_v11.cc`、`clean-v11/pmux_competition_v11.cc`，并由 patch 安装为 Yosys 的 `passes/opt/pmux_competition_opt.cc`。以下源码行号只适用于此摘要的版本。

# 1. 任务目标、验收依据与当前结论

本项目在官方 Yosys 0.69 上实现内建命令 `pmux_competition_opt`，在 `synth_intel` 中调用，对译码条件控制的多路选择结构进行通用逻辑化简。外部综合命令保持 `synth_intel -family cycloneiv`；候选版在内部加入优化 pass，并统一采用一份固定 ABC9 脚本。

赛题指南物理第 6～8 页要求说明算法原理、整体架构、模块划分和测试方案，另有现场答辩及团队分工要求。本文补充算法与工程说明；文末的成员身份、实际贡献和答辩分工仍须由三名成员据实填写。设计文档和答辩的最终评分由评委决定，不能由程序测试结果代替。

本次规则核对使用用户提供的《FPGA创新大赛问题汇总.xlsx》，工作簿 SHA-256 为 `dddf422c6945666f5e3548a8f582c363f3f3ab2051b88b7c7eb9251fd37a3c09`，以下坐标均指 `Sheet1`。较早答复与后续答复冲突时，按较新且更明确的答复实施，并保留未明确部分。

| 事项 | 采用的要求 | 来源 |
| --- | --- | --- |
| 工具与集成 | Yosys 0.69；新 pass 编入主程序，在 `synth_intel.cc` 调用 | C24、C37、C53 |
| 映射流程 | 使用内置 ABC9 和 Cyclone IV 流程；原版为未修改基线 | C53、C54 |
| 优化对象 | 结果以网表为准，可采用能优化相关逻辑的通用方法 | C34 |
| 面积 | 最终网表 LUT 加独立 NOT；全部九例逐例缩减率的算术平均；单例逻辑资源不可增加 | C25、C67、C115、C130 |
| DFF | 单独记录；较新答复说明 DFF 增加不作主要关注 | C130；同时保留较早 C21/C53 的约束背景 |
| 效率 | 每例相对原版的时间、内存增幅均不超过 5% 为满分范围；任一超过 15%，按该答复效率项不计分 | C105（对 B105 第 1 问的确认） |
| 测量范围 | 完整 Yosys 启动到结束；一次综合同时测时间和根 `yosys.exe` 峰值工作集 | C54、C87、C92、C133 |
| 计时实施 | 较新答复推荐 PowerShell `Measure-Command`；本文据此采用完整墙钟作为主判，CPU 作为辅证 | C101、C102；较早 C26/C41 使用 CPU 措辞 |
| 预热与复核 | 每例原版先预热十次，再原版→候选；不足一秒的小例超 5% 时可复核五次取平均 | C132 |
| ABC 限制 | 九例使用统一固定脚本；不修改其他 pass 或依赖程序以抵消新增开销 | C114、C130 |
| 交付 | 提交源码、修改的 ABC 脚本、可执行文件和依赖；相对路径，解包可启动 | C55、C131 |

| 事项 | 采用的要求 | 来源 |
| --- | --- | --- |
| 功能证明 | 原始 RTL 对 ABC 后的原生 `$lut` ILANG；可采用整设计证明；EQY 脚本自行编写 | C53、C54、C105 |
| 硬件与 eLinx | 本赛题无需 FPGA 硬件，也不要求 eLinx 工程和仿真报告 | C66、C71 |

截至 v21 已有证据表明：当前九个未修改输入的逻辑面积平均下降 20.06190452929184%，九例逻辑资源均不增加，实际 Windows 十八份网表与源 RTL 的 EQY 证明通过。按已经公开并固定的“追加五次复核不含首轮”规则，九例正式选定的时间和内存增幅均不超过 5%。

但是，test1 首轮存在明显耗时异常，复核是否包含首轮的官方文字尚不明确；因此本文不写“效率已无条件通过”或“赛项整体 PASS”。同日已新增独立 Linux/WSL EQY companion 包并通过迁移证明，详见第 9 节；测量环境条件、真实团队分工和现场答辩仍需完善。

# 2. 整体架构与综合调用流程

## 2.1 两版程序的比较关系

两版均基于官方 Yosys 0.69，源码基线提交为 `9f75ca1f9834a39a863915b5dae0c7b1e33533bc`，使用相同 ABC 二进制和对应平台相同的编译配置。候选版的改动包括新增 pass、构建注册、在 `synth_intel` 内调用该 pass，以及提供统一 ABC 脚本。基线不加入这些优化改动。

外部输入流程在两版之间采用同一种形状：

```
原始 RTL
  | read_verilog（按文件类型可加 -sv）
  ▼
synth_intel -family cycloneiv -top <top>
  ├── 原版：官方默认综合流程
  └── 候选：coarse 阶段 proc 后运行 pmux_competition_opt
            其余既有综合阶段继续执行
            map_luts 阶段仍使用 abc9，指定唯一固定脚本
  ▼
Cyclone IV 最终网表 ———→ LUT、NOT、DFF、总 cell 统计

独立形式导出链：同一综合流程运行至 map_cells 之前
  ▼
ABC9 后的原生 $lut / dffeas ILANG ———→ 原 RTL 对网表 EQY
```

集成 patch 在 `coarse` 标签的 `proc` 后加入 `run("pmux_competition_opt")`。这一位置使 Verilog 过程语句已转换为 `$mux`、`$pmux` 等内部单元，同时仍保留等式译码与字级运算结构，便于识别。优化 pass 不自行调用其他综合器，也不试跑多个综合流程选择结果。

候选 `map_luts` 调用为：

```
abc9 -lut 4 -W 300 -script +/intel/competition_abc_v11.abc
```

默认情形如带有 `-dff`，仍按 `synth_intel` 原有选项追加对应参数。固定脚本内容为：

```
&dch -f; &if -K 4 -a -C 4 -F 7 -A 1
```

这里保持 4 输入 LUT 目标。参数调整属于允许的 ABC 脚本调整；没有替换 ABC 二进制或修改 ABC 源码。九例共享该脚本，不按文件名、规模或网表特征选择不同脚本。程序中的 cell 名字排序仅用于确定遍历顺序，不以用例名字选择优化策略。

### 2.2 模块职责和数据接口

| 模块或数据结构 | 输入 | 输出/作用 | v21 源码位置 |
| --- | --- | --- | --- |
| pass 注册与入口 `PmuxCompetitionOpt` | 当前 Yosys 选区、模块及单元 | 内建命令、阶段调度与日志 | 298～310 |
| 快速可行性检查 | 已选择且未设 cell keep 的 mux/pmux | 无可改写对象时避免建立 SigMap；条件不足时跳过重型阶段 | 312～323、346～348、371～375 |
| `SigMap` | 模块连接和别名 | 统一比较同一信号的不同连线表达 | 323 及后续调用 |
| `facts` | `$eq` / `$logic_not` | → { → }，供条件替换 | 324～345 |
| `decode` | 满足更严格条件的 facts | → ( , )，供互斥性识别 | 325、344 |
| 条件常量替换 | mux/pmux 的 S、B 数据 | 仅替换对应 B 分支内可证明的信号位 | 349～370 |
| `factor_pmux_operators` | PMUX、decode、连接索引 | 共享 XOR/AND/OR 或满足成本门槛的 add/sub | 18～296 |
| 完整选择表与控制分组 | equality-decoded PMUX | 每一控制码的输出值、共用控制向量的一组目标 | 377～419 |
| 共享决策图搜索 | 预计算输出列、控制位排列 | 共享 bit-mux DAG、最小节点排列 | 421～487 |
| 成本判断与实体重建 | 最优 DAG、原 PMUX 估计 | 符合门槛才创建 `$mux` 并替换目标 | 488～514 |
| 测量与形式助手 | 固定工具、九例清单、源码及网表摘要 | 两条独立证据链，不把导出成功当作证明成功 | 独立 Python 脚本 |

## 3. 条件事实提取与数据替换

### 3.1 能提取什么事实

扫描模块内 `$eq` 和 `$logic_not`。对 `$logic_not(a)`，按 `a == 0` 建立事实。对等式，将完全常量的一侧放到 B 侧，只接受以下结构：

- 输出 Y 是一个有线信号位；A、B 位数一致。
- B 完全由已知的 0/1 构成，含 X/Z 的常量不作为事实来源。
- A 的常量位必须与 B 对应位一致；若同一有线位在 A 中重复出现，所有位置对该位的要求必须相同。

因此，选择信号 `s = (ctrl == 3'b101)` 为真时，可建立 `ctrl[2]=1, ctrl[1]=0, ctrl[0]=1`。该事实只在对应分支被选中的条件下成立，并不把控制信号全局改接为常量。

`facts` 对控制位宽没有 2～6 位限制；它可服务于只有普通 `$mux` 的模块或宽译码器。`decode` 是更严格的子集：控制宽度必须为 2～6，且事实数等于控制位数，排除控制向量中的常量位、重复位等不满足完整唯一编码的结构。只有后者用于操作数提取和共享树重建。

### 3.2 替换范围和正确性理由

对选中的、未设 cell keep 的 `$mux` / `$pmux`，检查 B 总宽度等于输出宽度乘选择分支数。逐一读取 `S[arm]` 对应的事实，只将该分支 B 中直接出现的、经 SigMap 归一后的信号位换为已知常量；不改默认输入 A，也不全局改线。

```
for each selected non-kept mux/pmux:
    require size(B) == width(Y) * size(S)
    for each arm:
        if S[arm] has conditional facts:
            for each data bit in B[arm]:
                if canonical(data bit) is in facts[S[arm]]:
                    replace this branch bit by the implied 0/1
```

小例：`y = (ctrl == 2'b10) ? {ctrl[1], d, ctrl[0]} : other`，其真分支可改为 `{1'b1, d, 1'b0}`。当选择条件为假时仍选择 `other`；未对 `other` 施加该条件。它不需要改变优先级 mux 的选择顺序。

宽译码条件下也必须先执行这一步，再判断是否进入重型阶段。v21 的 `decode.empty() || !has_pmux` 返回位于替换之后（371～374），不会因“只含 `$mux`”或“译码宽度大于六位”而漏掉合法替换。`substituted_bits` 记录执行的位替换次数，供回归检查，不作为面积收益的替代指标。

## 4. 跨 PMUX 分支的运算共享

### 4.1 为什么需要连接关系保护

若各 PMUX 分支分别计算同一种运算，选择后再计算一个共享运算，可能减少运算单元数量。例如：

```
原结构：  PMUX(a ^ k, b ^ k, c ^ k, d ^ k; select)
候选：    PMUX(a,     b,     c,     d;     select) ^ k
```

但不能仅凭代数式相似就删除分支运算单元：其输出可能被模块输出、其他单元或选择信号共同使用，也可能存在保留属性和位宽截断。因此，`factor_pmux_operators` 先建立所有模块连接的 driver/user 索引。模块输入作为不能删除的 driver，模块输出作为额外 user；未知单元的连接保守地同时视作输入和输出。

该函数在模块具有 `keep` 属性时直接返回。对内部可改写目标，它要求目标 PMUX 和分支运算单元均被选择，且没有 cell keep。分支运算输出必须逐位满足：

- 有且只有该运算单元驱动；不是常量、别名重复位或保留线位。
- 每个位只有一个使用单元，而且必须是当前 PMUX；不能再驱动模块输出或其他单元。
- 不同时作为当前 PMUX 的选择位。
- 运算完整 Y 向量与该 PMUX 数据分支一致；A/B/Y 的端口宽度及参数宽度均等于 PMUX 输出宽度。

只有确认全部原用途已被重接后，才删除旧的分支运算单元。函数维护索引的删除/新增，避免同轮嵌套改写继续使用陈旧的 driver/user 信息。

## 4.2 译码与遍历条件

运算共享只处理 2～6 位控制向量的 equality-decoded `$pmux`：所有选择信号必须来自同一控制向量的不同编码，不能重复编码，也不能混入原始任意选择位。因此，在已定义二进制控制值下，选择分支互斥。

函数对 PMUX 目标按名字排序，再沿 PMUX/mux/XOR/AND/OR/add/sub 的 A、B 数据依赖做深度优先遍历，以便先处理内部目标，再处理其外部使用者。此阶段检测组合依赖环并跳过涉及环的 factoring 目标。触发器不在遍历的运算集合内，属于叶子；不会把跨寄存器反馈误当成组合环。这里的环检测只描述该 factoring 阶段，不表示 pass 对任意组合环电路作了完整保证。

## 4.3 XOR、AND、OR 公共操作数提取

要求各数据分支由同一种 `$xor`、`$and` 或 `$or` 产生，至少有两个不同运算单元。由于这些操作可交换，公共操作数可出现在不同分支运算的 A 或 B 侧。找到共同的完整向量后，将剩余操作数送入原 PMUX，PMUX 输出再与公共向量运算一次。

如果默认输入 A 是实际值，则它也必须来自满足同样条件的运算单元，并参加公共操作数检查。若默认 A 是整向量 X，XOR 允许省去默认运算分支，因为 `X ^ common` 仍为 X。AND/OR 对整 X 默认不做该提取，例如 `X & 0` 可能成为确定的 0，这不应在该变换中被轻率使用。混合已知位和 X 的默认值不会自动当作“整 X 默认”。

该阶段以“多个同类运算变为一个、保留一个数据 PMUX”为结构收益依据，没有运行 ABC 进行探测，也没有单独预测最终 LUT 必然减少。因此，是否满足比赛的单例逻辑非增仍由完整综合后的网表验证。

## 4.4 add/sub 字级运算共享

加减法共享保留运算方向：对于减法，A/B 不可交换。全部分支要求同一 `$add` 或 `$sub` 类型，A_SIGNED 与 B_SIGNED 参数也必须逐分支一致。复用后的共享运算显式保留这些参数。数据宽度至少四位，并要求 A/B/Y 的完整宽度一致，不跨越隐式扩展或截断边界。

每个操作数位都必须是有线位，单个操作数内部的位必须各不相同；A 与 B 不得为同一整个向量。此处是保守地避开常量、重复位和显然可简化的低成本运算，不能解释为数学意义上证明了这些位彼此独立。按不同 `(A,B)` 向量对估计需共享的运算个数；加法允许把交换的 `(B,A)` 当作同一对，减法不允许这样合并。

有公共 A 时，仅为不同 B 建立一个 PMUX；有公共 B 时反之；两侧均变化时建立两个输入 PMUX。若两侧均相同，则已有运算表达本就相同，不在这里计作新的共享收益。

小例，保持方向的减法：

```
PMUX(base-a, base-b, base-c, base-d; select)
    → base - PMUX(a, b, c, d; select)
```

有实际默认值时，默认也必须由合格的同类运算产生。整 X 默认仅在所有 `2^n` 个二进制控制编码都有不同 equality 分支、默认在这些编码下不被选择时允许省去；不利用 reset/enable 条件或用例约束来宣称默认不可达。稀疏编码的 add/sub 整 X 默认会跳过；混合 X 默认不走该特例。

## 4.5 算术共享的成本门槛

令输出位宽为 W，显式 PMUX 分支数为 A，不同操作数对数为 I，所需输入 PMUX 数为 M（1 或 2）。代码使用整数除法估计：

```
arithmetic_luts = W + floor((W - 1) / 2)
pmux_luts       = W * (floor((A + 1) / 2)
                       + floor((floor(A / 2) + 2) / 3))
estimated_gain  = (I - 1) * arithmetic_luts - (M - 1) * pmux_luts
执行门槛：estimated_gain >= max(2, floor(W / 2))
```

`M-1` 体现原输出 PMUX 被移走、其中一个输入 PMUX 可与其成本抵消的粗略估计。共享等式译码器没有被计入收益，避免把本来就共用的译码重复算作节省。

例如 W=8、A=3、I=4、M=1 时，估计算术成本为 11，估计收益为 33，门槛为 4。这个数只是是否尝试结构变换的门槛，并非最终将精确减少 33 个 LUT。ABC 的重写、进位映射、PMUX 分解和跨输出共享都会影响实际结果。

# 5. 完整默认表与跨输出共享决策图

## 5.1 从稀疏 equality PMUX 得到完整表

在运算共享之后，收集仍存在的、选中且未设 cell keep 的 PMUX。要求各选择信号属于同一 2～6 位控制向量，各编码唯一。建立 `table[0 … 2^n-1]`：显式编码填对应 B 数据，未列出的编码填原默认 A。只有编码全部列满时，默认值不参与这些二进制编码的输出。

例如两位控制、分支 0→a、2→b、默认→d，完整表为 `[a,d,b,d]`。这一步没有把所有缺省项武断填零，也没有把 X 改成任意有利常量；表项保留归一后的原数据位。

按完整 `SigSpec ctrl` 相等分组，同一组所有目标、所有输出位共同参与一张共享图。不同向量、不同位顺序即使可能存在语义关系，也不在该规则里自动合并。

## 5.2 按 Shannon 分解建立 bit-mux DAG

每个输出位是关于 n 个控制位的选择函数，叶子是一个原始数据位或常量；叶子不是整份 RTL 真值表，只枚举控制编码下选择哪一数据位。对一个控制位次序进行递归分解：低分支保持该位为 0，高分支置为 1，生成 `mux(lo, hi, selector)`。

```
build(column, code, depth):
    if depth == n:
        return column[code]
    lo = build(column, code, depth + 1)
    hi = build(column, code | (1 << order[depth]), depth + 1)
    if lo == hi:
        return lo
    key = (selector[order[depth]], lo, hi)
    return memoized_or_new_node(key)
```

若 lo 与 hi 相同，省略该选择节点；若多个输出位或多个 PMUX 子树具有相同 `(selector,lo,hi)`，使用同一节点。节点只在低、高子图完成后创建，形成可直接按顺序实体化的 DAG。普通 `$mux` 中 0 选择 A、1 选择 B 的语义与这里一致。

小例：完整列 `[a,b,a,b]` 只依赖 ctrl[0]，ctrl[1] 两侧子图相同而被消去，最终只需一个 `mux(a,b,ctrl[0])`。两路输出若含有同一中间选择子图，也可复用同一 bit-mux 节点。

### 5.3 选择位排列搜索与确定性

控制位数限于 n=2～6。遍历 n! 个排列，最多 `6! = 720`，每个排列为整个控制组建立共享 DAG，以总 bit-mux 节点数为代价。只有节点数严格更小时才替换当前最优方案，平局保留首次遇到的排列，因此结果选择规则固定。

v21 将目标→输出位→控制编码的叶子列预先计算一次，先注册控制位，再按该顺序注册数据叶子。递归不再为每个内部节点分配 active_codes、lo、hi 向量，而用整数 `code` 记录已经选择的高分支。

等价条件是：order 为各控制位的一个排列；递归深度前的位已设置且后面的位尚未设置；先递归低分支，再递归高分支。这与原升序编码集合逐位分割的访问顺序相同，因此叶子编号、内部节点插入顺序、输出编号以及首次最优排列保持一致。离线整数模型对 2～6 位、常量/重复叶、多目标等 381 组输入、5,906 个排列试验进行了逐图对照，并用故意错误的高位索引确认对照程序能发现差异。

上述图一致不等于 ILANG 文件逐字节一致：`NEW_ID` 可能包含源码行号，改动源码位置会影响对象名称。v21 因此重新进行了实际输出的形式证明，未以图对照替代 EQY。

### 5.4 重建成本、复杂度及限制

每个目标原始 PMUX 的估计为：

```
estimate(target) = target.width *
    (floor((explicit_arms + 1) / 2)
     + floor((floor(explicit_arms / 2) + 2) / 3))
```

组内相加为旧结构估计成本。使用显式分支数而非补全后的 `2^n` 个编码，避免为原本很稀疏的 PMUX 虚增基线估计。仅当最优共享节点数不大于该估计时，创建实际 `$mux`、重接原目标输出并删除原 PMUX。

若该组全部目标的输出位数之和为 S，单次排列最多考察约 `S*(2^n-1)` 个递归节点，搜索工作随 `n! * S * 2^n` 增长，memo 操作还有有序映射开销。预计算列需要 `S*2^n` 个整数；试验图的空间亦受该规模限制。n≤6 限制了指数和阶乘的控制维度，但不保证任意超宽输出或任意大型模块都在固定时间/内存内。

“bit-mux 节点数”与最终 4-LUT 数不是严格一一对应。成本门槛只是避免明显不合算结构，最终非增仍必须按完整映射网表验证，不承诺任意新电路都减少资源。

## 6. 安全条件、未覆盖语义与实现边界

| 条件 | 当前实际保护 | 不能扩大解释为 |
| --- | --- | --- |
| 选择范围 | 改写目标限 selected mux/pmux；factoring 的 producer 也必须 selected | 没有选择任何对象也会进行全局优化 |
| blackbox | pass 入口跳过 blackbox 模块 | 为任意未知单元提供功能模型 |
| cell keep | 条件替换、factoring、共享树均跳过带 keep 的目标；factoring 还保护 producer keep | 所有标准综合阶段均不会处理 keep 对象 |
| module/wire keep | factoring 跳过 module keep，且不删除保留线上的 producer；其 driver/user 索引包含端口 | 条件替换和共享树拥有同样的 module/wire keep guard；这两阶段实际只检查目标 cell keep |

| 条件 | 当前实际保护 | 不能扩大解释为 |
| --- | --- | --- |
| 多用户与多驱动 | factoring 要求完整 producer 输出只由该 producer 驱动、仅被当前 PMUX 使用 | 其他阶段对任意复杂网络完成了全局单驱动证明 |
| 位宽与 signed | factoring 要求完整等宽；add/sub 保留并逐分支比较符号参数 | 支持任意混合宽度、截断、符号扩展的运算共享 |
| 优先级 | 不把普通 priority mux 当作互斥 PMUX 重排；条件替换保留选择结构 | 对任意原始 one-hot 输入直接假设互斥 |
| 默认与 X | 保留未列出编码的 A；对 X 默认采用前述分操作规则 | 在任意四态仿真、未知控制输入下都有完整验证保证 |
| 寄存器与反馈 | 自定义 pass 不创建/删除/改接触发器单元，factoring 将其作为叶子 | 最终网表 DFF 数必定相同；后续标准综合可能进一步简化 DFF |
| 成本估计 | 算术及共享树有固定启发式门槛；无综合探测 | 任意案例最终 LUT 保证不增加 |

52 项通用运算回归使用定义良好的二进制输入，并将源设计字面 X 作为 don't-care（`-ignore_gold_x`）；不应将它称为所有四态 Verilog 行为的穷尽证明。实际九例另有整设计 EQY。对任意多时钟、锁存器、存储器、新原语或复杂异步控制网络，必须扩展模型与审查后再给出新的验证结论。

# 7. 功能验证方案与证据链

## 7.1 三层验证

第一层是结构回归。52 项通用运算边界检查覆盖 XOR 交换操作数、AND/OR 默认值、整 X/混合 X、外部用户、keep、宽度不符、缺少公共操作数、嵌套 PMUX，以及 add/sub 的公共 A、公共 B、两侧变化、signed 不匹配、常量/重复位、成本边界、混合控制、原始选择信号和重复编码等。每例检查是否应发生 factoring，并将 pass 前后的设计进行 SAT 等价验证。

第二层是六项快速路径回归：无 mux、有 PMUX 但无 facts、八位译码下仍执行数据替换，以及仅有 mux、译码宽度分别为 1/3/8 的情形。它们检查替换位数、输出常量、没有多余的 factoring/共享树阶段，并进行 SAT。故意将输出改为恒零的负对照必须被检出。

第三层是九例整设计验证，原版与候选各九项，共十八项。使用完整 `synth_intel` 统计最终资源；另一运行用 `-run begin:map_cells` 在 ABC9 后、最终 Cyclone IV LUT 映射前导出原生 `$lut` ILANG，再续跑 `-run map_cells:`。检查分段续跑和独立完整流程的最终各类 cell 数一致，避免导出流程与实际资源统计流程脱节。

## 7.2 EQY 对象、模型和结果判定

gold 为原始 RTL；gate 为实际导出的 post-ABC ILANG。gate 侧先用 `lut2mux -word` 展开 LUT，再加载受审查的 `dffeas` 功能模型。模型保留动态、低有效异步 `clrn`，没有把它固定为解除复位。

在采用模型前，脚本逐个检查原生网表的单元类型和触发器控制：仅接受已支持类型；检查 `prn=1`、`aload=0`、`asdata=0`、`sclr=0`、`sload=0`，以及 d/clk/clrn/ena/q 的标量性、控制信号和允许的 power_up 参数。遇到不支持的配置报告错误，而非默默使用错误模型。

EQY 使用 SAT depth 5 和 SBY/ABC PDR 策略。输出 PASS 必须同时有正常退出和 PASS 标记；存在确切反例记录则判 FAIL；超时或未获得明确证明为 UNKNOWN，环境或版本异常为 ERROR。不会把 UNKNOWN 当作通过。

test2 另外构造恒零错误输出网表，必须出现确切 counterexample，验证链能够拒绝错误电路。负对照预期的 FAIL 是校验成功，不应计入正向证明的失败。

Linux 私有候选、Ubuntu 稳定 current 部署默认入口分别完成十八项证明；Windows 则对本次 PE 程序实际导出的十八份 ILANG 再完成独立证明。三条链各自覆盖九例两角色，不能叠加声称为 54 个不同测试用例。Windows ILANG 原始 CRLF 摘要保留，仅为 Linux 工具制作可逆换行归一副本并验证往返一致。

### 7.3 证据绑定

每例登记输入源码、综合脚本、工具、ILANG、最终网表与日志 SHA-256。Windows 实际形式证明同时绑定原测量 report、Windows manifest、PE、ABC 脚本与每份 ILANG，避免用旧候选的证明覆盖新候选成绩。

| 关键对象 | SHA-256 |
| --- | --- |
| v21 自定义 pass | `0231002c240a080e7eebe1734b9dab49f2db78159c38a68769c885599eb43678` |
| 统一 ABC 脚本 | `bb92697ba7adcd8f97827885daaf8ef501d49d79428b54e51122fbffc45c5fa7` |
| 集成 patch | `fa6123b1ef9ea137576cd8e24333d6ae842642d7252a01aa603ace4429836ee8` |
| Windows 候选 PE | `17508a2d928a97eba82086f0068a5d1b8e1d2bff2d454d6da23b210648f52a99` |
| Windows 原版 PE | `2530e581996e1c088d5a35b85c9430efcbaef30d8ce420ed660427a38e0e584f` |
| 两版相同 ABC 二进制 | `b17ea3f805418a28dc29250721be04ff2ec708fd8226e65a0815bbd6d11c9ad8` |
| Windows 原始测量 report | `a2d12cfe60767e44bbbb8d025bea74a1b14f7811e6e45d4ff085874b8a31f7c8` |
| Windows 实际网表形式报告 | `f7fd0e3aab8d5f45b5458802e5957ab847018cf6bdc5f924a488852886038dde` |
| 部署后 Linux 默认入口形式报告 | `129d36f907819a12a3b1930ef5e3965394f2b81a606650fab9a0714ab974279d` |

历史 raw 测量报告中的 `formal_status=PENDING` 是生成时的状态，保留原字节；后续正式报告通过摘要补充证明，不倒改原始记录。

## 8. 性能测量设计、当前成绩与异常

### 8.1 固定测量协议

原版和候选在同一 Windows 环境运行，使用相同测量器。每例先启动原版十次预热；正式首轮按原版→候选。墙钟从 `CreateProcessW` 调用前计到根进程结束，包含程序创建、绑定 Job Object、启动、RTL 读取、完整综合、等待 ABC 和网表写出；同一次综合读取根 `yosys.exe` 的 PeakWorkingSetSize。包含 ABC 后代的 CPU 时间另行记录作辅助。

C101/C102 推荐的 `Measure-Command` 测量墙钟，本文据此将墙钟作为主判；这是一项公开的计时实施解释。较早 C26/C41 仍使用 CPU 措辞，现场应遵循官方统一口径，不能声称两种指标等价或二者都已全部过线。当前选定样本的 Job CPU 在 test1 为 **+8.75%**、balanced 为 **+17.647059%**；Windows CPU 计数有明显量化步长，本文保留这些

原值，不用量化理由将其改写为通过。

首轮原版不足一秒且候选时间或内存增幅超过 5% 时，追加五对原版→候选，分别对五次角色测量取算术平均，再求候选与原版均值之比；首轮保存但不纳入该五次均值。原版超过十秒不复核；一至十秒区间按单次处理并标明官方未详述。

这是预先固定的实现解释，不是 FAQ 对全部细节的逐字规定。C132 还要求后台统一和“杀软关闭”；脚本没有改变任何安全设置，现有证据未确认测量时杀软已关闭，因此不能声称这一环境条件已严格验收。

## 8.2 九例资源和正式选定效率

逻辑数 L 为最终 Cyclone IV LUT 数加独立 NOT 数；每例缩减率为 `(L_baseline-L_candidate)/L_baseline`，九例等权平均。DFF 单独列出。

| 用例 | 逻辑原版→候选 | 降幅 | DFF 原版→候选 | 正式墙钟变化 | 正式峰值内存变化 | 选定样本 |
|---|---|---|---|---|---|---|
| test1 | 288→227 | 21.180556% | 222→222 | -0.350998% | -1.161697% | 追加五对均值 |
| test2 | 25→15 | 40.000000% | 0→0 | -2.643562% | -0.025484% | 首轮 |
| test3 | 13→13 | 0.000000% | 17→12 | -3.108560% | -0.738294% | 追加五对均值 |
| test4 | 36→27 | 25.000000% | 10→10 | -4.268916% | -1.819499% | 追加五对均值 |
| balanced | 426→356 | 16.431925% | 12→12 | -2.143691% | -1.540226% | 首轮 |
| pipeline | 450→348 | 22.666667% | 24→24 | -2.895130% | -3.544385% | 首轮 |
| crossbar | 408→285 | 30.147059% | 10→10 | -13.022651% | -9.445178% | 首轮 |
| feedback | 368→337 | 8.423913% | 16→16 | -2.954609% | +0.264364% | 首轮 |
| nested | 413→344 | 16.707022% | 12→12 | -1.512720% | -2.900213% | 首轮 |

九例面积平均下降 **20.06190452929184%**，超过 20% 目标的余量仅 **0.06190452929184 个百分点**。表中正式选定值全部满足时间和内存增幅不超过 5%；它不是“所有原始运行均不超过 5%”的结论。

全部候选首轮和复核样本（包含未进入均值的首轮）最大墙钟为 **0.6765548 秒**，最大根进程峰值工作集 **24,612,864 字节**。当前九例输出总 cell 最大为原版 test1 的 510，均小于 1000，实测满足 60 秒/2 GB。此结果不证明任意≤1000节点输入都满足绝对上限。

## 8.3 必须保留的首轮异常与口径敏感性

本轮 90 次原版预热、48 次正式首轮及追加复核，共 138 条原样本均保留。正常运行均退出 0，记录中 Job 内活跃后代为 0，无限制终止，完成最终峰值读取。未记录 `CloseHandle` 的系统返回值，故只报告程序 finally 关闭路径和无活跃后代事实，不声称独立核验了句柄关闭成功字段。

三例首轮超过 5%：test1 为 **+127.518916%**，test3 为 **+6.044197%**，test4 为 **+13.940824%**。test1 首轮原版 0.2973620 秒、候选 0.6765548 秒；追加五次均值分别为 0.30770016 秒、0.30662014 秒。没有删除、替换或筛选首轮异常。

FAQ C132 的“复核5次取平均”未说明首轮是否计入。对同一已保存数据的替代汇总如下，不改变任何测量样本：

| 用例 | 追加五次、不含首轮（当前规则） | 首轮加前四次，共五次 | 首轮加五次，共六次 |
|---|---|---|---|
| test1 | -0.350998% | +24.682130% | +20.360601% |
| test3 | -3.108560% | -1.296126% | -1.710473% |
| test4 | -4.268916% | -0.107417% | -1.433749% |

因此 test1 的效率判定对解释敏感：若官方要求平均包含首轮，该次证据会超过 15%。现有证据不能确定首轮异常由自定义算法、系统调度、加载、缓存或后台软件中的哪项造成。不能凭复核正常就宣称原因已经消除；下一轮优化和统一条件复测应继续针对这一点，不能靠改基线、删样本或事后改变复核规则过线。

## 9. 交付、复现方式与尚待完善项

当前 Windows 包携带两版 PE、相同 ABC、必要 DLL/share、隔离 Python、九例输入、相对入口、测量器、源码/patch/ABC 脚本、已有证明报告和日志归档。在 ASCII 路径解压后，`run_cases.cmd` 使用 `%~dp0` 动态定位包，不依赖构建机器的绝对路径，也不需要外部 Python。最终 ZIP 换目录、干净 PATH 的十八次完整综合 smoke 检查通过。

v21 最终 Windows 包为 `FPGA-topic2-v21-Windows-0.69-20261027.zip`，SHA-256 为 `931c68d5aeac5e8e3acb331c733becc07733958e4c7decb8c728414c496a34c6`。ZIP 摘要和迁移检查保存在旁边 sidecar，避免把包含最终 ZIP 摘要的文件再放回同一 ZIP。本文和下述独立证明包在该 ZIP 冻结后生成，须作为配套材料一同交付；原 ZIP 没有被覆盖。

新增 companion 为 `FPGA-topic2-v21-EQY-Linux-0.69-20261027.tar.gz`，43,472,694 字节，SHA-256 为 `1e310f0210e364649be7905e18b7e9ba26b1b81af59ce5002ffd0d0b8b371501`。在 Linux x86_64 的新 ASCII 目录解包后，运行 `./run_verify.sh`；每轮创建新的结果目录。包内含隔离 CPython 3.10、Click、原版 Yosys 0.69/ABC、EQY 0.69、SBY 0.68、模型、GNU 工具及私有 ELF loader/共享库。四个冻结 helper、九例输入、原始 Windows 测量报告和十八份原始网表均保持原字节。此入口只证明实际 Windows 网表，Linux manifest 中的 candidate 明确是原版解析器别名，不表示重新综合了 Linux 优化版。

同日从归档解出到全新 `/tmp` 目录，在干净 PATH、无效外部 Python/ABC/share 设置下运行，十八项正向 EQY 均 PASS，故意错误输出成功产生确定反例。证明报告 SHA-256 为 `a2e05c107f855e47463977aaaf39c0a854ce034f10d2757f3bcfa8fccc9f53d4`；全部活跃输入与工具指纹路径位于新包目录。独立审计重新核对 1,442 项 payload 摘要和入口执行位。首次 r3 尝试曾因私有 loader 改变资源查找位置失败；保留失败记录，修正相对 share/ABC 布局后，r4 完整重跑通过，未修改 Yosys 二进制或证明策略。

仍须区分以下限制：

1. **独立形式验证已有 Linux/WSL 方案，仍有平台边界。** 原 Windows ZIP 内的旧 helper 单独使用仍需要外部证明环境；新增 companion 已在新目录独立复现十八项证明和负对照，应与原 ZIP 一同提交。它需要 Linux x86_64 或兼容 WSL2 Linux 内核、`/bin/sh`、`/proc`、`/dev/null` 和可写存储，不等于普通 Windows 原生 EQY，也未验证所有发行版、内核或现场环境。C131 的交付要求与现场采用的平台仍须按官方验收执行。
2. **解压路径限制。** 当前 runner 明确要求 ASCII 路径，中文用户目录直接解压可能不能启动。相对路径迁移通过不能解释为任意 Unicode 路径均通过。
3. **首轮异常及复核解释。** test1 问题尚未从算法或运行机制上完全消除，官方对是否包含首轮的解释仍影响结论。

4. **环境与泛化。** 当前九例、同一台 Windows 环境的通过不能保证未来追加用例或现场机器；原始样本异常说明仍需留出波动余量。当前面积余量也很小。
5. **属性与模型边界。** module/wire keep 的保护按第 6 节逐阶段描述，不应泛称所有阶段保护完全一致；新增原语或时序控制必须先扩展模型检查。
6. **构建事实。** 本轮使用匹配 Yosys 0.69 编译对象增量构建，并重新综合、重新形式证明。未做本轮完全清洁重建，也未声称清空系统页缓存；这不是已发现的赛题明文缺项，但应如实说明工程复现范围。
7. **文档和答辩。** 本稿须与最终优化版本同步，经团队核对成员贡献、算法讲解和现场演示步骤后定稿。任何新候选均需绑定自己的源码、PE、脚本、网表和新报告，不能沿用 v21 结果。

Git HEAD `2791affaa80b9bdf637cae5f60511056aec6d8ab` 只是取代码时的上下文；本轮本地修订未提交、未推送。Git 同步是否完成与比赛算法是否满足指标是两件事，不将推送列为 PDF 明文性能要求。

## 10. 三人实际分工与答辩核对（待团队填写）

不能根据聊天记录推断每个人的实际工作。请填写能与提交记录、笔记、实验报告或现场讲解对应的真实贡献，不把下表当作已经完成的分工证明。

| 成员真实姓名/学号 | 已完成的具体工作 | 可核对的文件/提交/实验记录 | 负责讲解的算法或工程部分 | 现场演示与问答职责 |
|---|---|---|---|---|
| 成员一：待填 | 待据实填写 | 待填 | 待填 | 待填 |
| 成员二：待填 | 待据实填写 | 待填 | 待填 | 待填 |
| 成员三：待填 | 待据实填写 | 待填 | 待填 | 待填 |

答辩至少应能够由团队自行解释：为何只能在被选分支替换控制位；为何 equality 编码能证明互斥；如何处理真实默认值与 X；为什么多用户、位宽和 signed 不一致必须跳过；720 排列搜索如何限制规模；启发式成本为何不能保证最终 LUT；EQY 正向与错误负对照分别证明了什么；为何完整启动墙钟和根进程峰值是本次口径；test1 首轮异常及复核规则的局限。

现场演示步骤与备选方案、每人负责的问答范围、能否在现场环境完成构建/运行和形式重跑，均需团队实际演练并记录。本文不把尚未进行的演练写成已通过。

## 附录：主要文件与阅读顺序

| 顺序 | 文件/位置 | 用途 |
|---|---|---|
| 1 | `src/pmux_competition_v11.cc`（v21 摘要见开头） | 完整自定义 pass；按第 2～6 节对照源码 |
| 2 | `source/synth-intel-competition.patch` | 注册 pass、安装固定脚本和接入 synth_intel 的全部改动 |
| 3 | `cases_current.json` | 九例 top、相对路径和输入原始字节摘要 |
| 4 | `run_current_cases.py`、`windows_metrics.py`、`README_PROTOCOL.md` | 固定测量/复核协议和同次墙钟、根峰值测量 |
| 5 | `formal/tools/verify_current_cases.py`、`verify_windows_current.py`；新增 Linux/WSL companion | 原 Windows ZIP 中的 helper 仍依赖外部工具；companion 已携带工具并迁移复现十八项证明和负对照 |
| 6 | `formal/tools/test_pmux_operators.py`、`test_pmux_fastpaths.py` | 通用结构、保护条件和快速路径回归 |
| 7 | `report.json`、`formal/*report.json`、两个完整 proof ZIP | 原始性能数据、正负形式结果及日志 |
| 8 | `validation/current-v21-20261027/summary.json`、ZIP sidecar | 部署后状态、工具身份和最终包的边界 |

本报告使用冻结的 v21 综合、测量、形式报告以及同日独立证明包的迁移运行与审计记录；未借新增 companion 改写原始性能成绩。后续版本应更新源码行号、身份摘要、实际成绩和未解决项，并保留旧版本不利记录。